

✓ Experiment No. 8

Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

The experiment aims to implement optical-flow-based motion analysis for real-time object tracking using Python and OpenCV.

Aim

To implement an optical flow-based motion analysis system for real-time object tracking and evaluate its effectiveness in detecting and tracking moving objects in video sequences using Python and OpenCV.

✓ Requirements

- Python
- OpenCV (cv2)
- NumPy
- Matplotlib
- Jupyter Notebook / Google Colab / VS Code
- Knowledge of Lucas-Kanade and Farneback Optical Flow

```
import cv2
import numpy as np
import math
import matplotlib.pyplot as plt

# =====
# COLLAGE FUNCTION
# =====

def create_collage(images, titles, cols=2):

    if len(images) == 0:
        return None

    width = 400
    height = 300

    resized = []

    for image, title in zip(images, titles):

        image = cv2.resize(image, (width, height))

        cv2.rectangle(
            image,
            (0, 0),
            (width, 35),
            (0, 0, 0),
            -1
        )

        cv2.putText(
            image,
            title,
            (10, 25),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.65,
            (255, 255, 255),
            2
        )

        resized.append(image)

    rows = math.ceil(len(resized) / cols)

    collage = np.zeros(
        (
            rows * height,
```

```

        cols * width,
        3
    ),
    dtype=np.uint8
)

for i, image in enumerate(resized):

    row = i // cols
    col = i % cols

    collage[
        row * height:(row + 1) * height,
        col * width:(col + 1) * width
    ] = image

    return collage

# =====
# DISPLAY FUNCTION
# =====

def display_result(image, title="Result"):

    if image is None:
        return

    plt.close("all")

    plt.figure(figsize=(10, 7))

    image_rgb = cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )

    plt.imshow(image_rgb)
    plt.axis("off")

    plt.title(
        title,
        fontsize=12,
        pad=8
    )

    plt.tight_layout()
    plt.show()

    plt.close("all")

```

PART 1 — Lucas-Kanade

```

# =====
# PART 1: LUCAS-KANADE SPARSE OPTICAL FLOW
# =====

def lucas_kanade_tracking(video_path):

    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Could not open video.")
        return

    ret, old_frame = cap.read()

    if not ret:
        print("Error: Could not read video.")
        cap.release()
        return

    old_gray = cv2.cvtColor(
        old_frame,
        cv2.COLOR_BGR2GRAY
    )

```

```

# Shi-Tomasi parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Lucas-Kanade parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

# Detect feature points
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

mask = np.zeros_like(old_frame)

total_distance = 0
frame_count = 0

result_images = []
result_titles = []

total_frames = int(
    cap.get(cv2.CAP_PROP_FRAME_COUNT)
)

sample_frames = np.linspace(
    1,
    max(1, total_frames - 1),
    4
).astype(int)

sample_index = 0

print("Lucas-Kanade Optical Flow Processing...")

while True:

    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    if p0 is not None and len(p0) > 0:

        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )

        if p1 is not None and st is not None:

            good_new = p1[st == 1]
            good_old = p0[st == 1]

```

```

distances = []

for new, old in zip(
    good_new,
    good_old
):

    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    dx = x_new - x_old
    dy = y_new - y_old

    magnitude = math.sqrt(
        dx * dx + dy * dy
    )

    distances.append(magnitude)
    total_distance += magnitude

# Trajectory
cv2.line(
    mask,
    (int(x_old), int(y_old)),
    (int(x_new), int(y_new)),
    (0, 255, 0),
    2
)

# Feature point
cv2.circle(
    frame,
    (int(x_new), int(y_new)),
    4,
    (0, 0, 255),
    -1
)

# Motion vector
cv2.arrowedLine(
    frame,
    (int(x_old), int(y_old)),
    (int(x_new), int(y_new)),
    (255, 0, 0),
    2
)

if len(distances) > 0:

    avg_magnitude = np.mean(distances)

    dx_avg = np.mean(
        good_new[:, 0] -
        good_old[:, 0]
    )

    dy_avg = np.mean(
        good_new[:, 1] -
        good_old[:, 1]
    )

    angle = math.degrees(
        math.atan2(
            dy_avg,
            dx_avg
        )
    )

    if abs(dx_avg) < 1 and abs(dy_avg) < 1:
        direction = "Stationary"

    elif abs(dx_avg) > abs(dy_avg):

        if dx_avg > 0:
            direction = "Right"
        else:
            direction = "Left"

```

```

else:

    if dy_avg > 0:
        direction = "Down"
    else:
        direction = "Up"

# Information
cv2.putText(
    frame,
    "Lucas-Kanade",
    (15, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
    2
)

cv2.putText(
    frame,
    f"Points: {len(good_new)}",
    (15, 60),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

cv2.putText(
    frame,
    f"Magnitude: {avg_magnitude:.2f}",
    (15, 90),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

cv2.putText(
    frame,
    f"Direction: {direction}",
    (15, 120),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

cv2.putText(
    frame,
    f"Angle: {angle:.2f}",
    (15, 150),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

p0 = good_new.reshape(-1, 1, 2)

else:
    p0 = None

else:
    p0 = None

output = cv2.add(
    frame,
    mask
)

frame_count += 1

if (
    sample_index < len(sample_frames)
    and frame_count >= sample_frames[sample_index]

```

```

):

    result_images.append(
        output.copy()
    )

    result_titles.append(
        f"Frame {frame_count}"
    )

    sample_index += 1

old_gray = frame_gray.copy()

if p0 is None or len(p0) < 10:

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )

cap.release()

collage = create_collage(
    result_images,
    result_titles,
    cols=2
)

print("\nLucas-Kanade Completed")
print("Frames processed:", frame_count)
print(
    "Total motion:",
    round(total_distance, 2)
)

if collage is not None:

    display_result(
        collage,
        "Lucas-Kanade Optical Flow"
    )

# =====
# RUN PART 1
# =====

video_path = "/content/video - Trim.mp4"

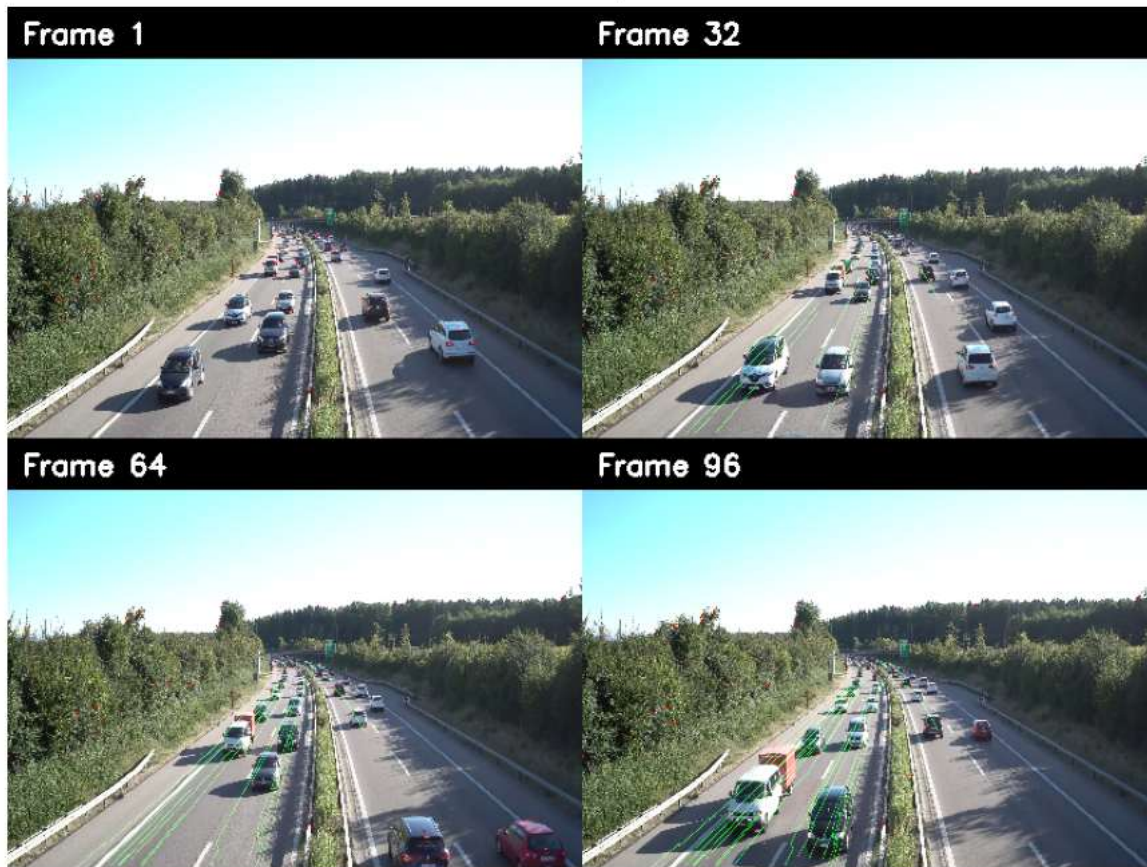
lucas_kanade_tracking(video_path)

```

Lucas-Kanade Optical Flow Processing...

Lucas-Kanade Completed
Frames processed: 96
Total motion: 14494.03

Lucas-Kanade Optical Flow



PART 2 — Farneback

```
# =====  
# PART 2: FARNEBACK DENSE OPTICAL FLOW  
# =====  
  
def farneback_tracking(video_path):  
  
    cap = cv2.VideoCapture(video_path)  
  
    if not cap.isOpened():  
        print("Error: Could not open video.")  
        return  
  
    ret, first_frame = cap.read()  
  
    if not ret:  
        print("Error: Could not read video.")  
        cap.release()  
        return  
  
    previous_gray = cv2.cvtColor(  
        first_frame,  
        cv2.COLOR_BGR2GRAY  
    )  
  
    frame_count = 0  
    total_motion = 0  
  
    result_images = []  
    result_titles = []  
  
    total_frames = int(
```

```

        cap.get(cv2.CAP_PROP_FRAME_COUNT)
    )

    sample_frames = np.linspace(
        1,
        max(1, total_frames - 1),
        4
    ).astype(int)

    sample_index = 0

    print("Farneback Optical Flow Processing...")

    while True:

        ret, frame = cap.read()

        if not ret:
            break

        current_gray = cv2.cvtColor(
            frame,
            cv2.COLOR_BGR2GRAY
        )

        # Calculate dense optical flow
        flow = cv2.calcOpticalFlowFarneback(
            previous_gray,
            current_gray,
            None,
            0.5,
            3,
            15,
            3,
            5,
            1.2,
            0
        )

        # Magnitude and angle
        magnitude, angle = cv2.cartToPolar(
            flow[..., 0],
            flow[..., 1]
        )

        avg_magnitude = np.mean(
            magnitude
        )

        total_motion += avg_magnitude
        frame_count += 1

        # HSV visualization
        hsv = np.zeros_like(frame)

        hsv[..., 1] = 255

        hsv[..., 0] = (
            angle * 180 / np.pi / 2
        )

        hsv[..., 2] = cv2.normalize(
            magnitude,
            None,
            0,
            255,
            cv2.NORM_MINMAX
        )

        dense_flow = cv2.cvtColor(
            hsv,
            cv2.COLOR_HSV2BGR
        )

        # Draw motion vectors
        step = 20

```

```

for y in range(
    0,
    frame.shape[0],
    step
):
    for x in range(
        0,
        frame.shape[1],
        step
    ):
        fx = flow[y, x, 0]
        fy = flow[y, x, 1]

        if magnitude[y, x] > 1:
            cv2.arrowedLine(
                dense_flow,
                (x, y),
                (
                    int(x + fx),
                    int(y + fy)
                ),
                (255, 255, 255),
                1,
                tiplength=0.3
            )

# Information
cv2.putText(
    dense_flow,
    "Farneback Dense Flow",
    (15, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (255, 255, 255),
    2
)

cv2.putText(
    dense_flow,
    f"Motion: {avg_magnitude:.2f}",
    (15, 60),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

cv2.putText(
    dense_flow,
    f"Frame: {frame_count}",
    (15, 90),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 255),
    2
)

# Save selected frames
if (
    sample_index < len(sample_frames)
    and frame_count >= sample_frames[sample_index]
):
    result_images.append(
        dense_flow.copy()
    )

    result_titles.append(
        f"Frame {frame_count}"
    )

    sample_index += 1

previous_gray = current_gray.copy()

```

```

cap.release()

# Create collage
collage = create_collage(
    result_images,
    result_titles,
    cols=2
)

# Calculate average motion
if frame_count > 0:

    average_motion = (
        total_motion / frame_count
    )

else:

    average_motion = 0

print("\nFarneback Completed")
print("Frames processed:", frame_count)
print(
    "Average motion:",
    round(average_motion, 2)
)

if collage is not None:

    display_result(
        collage,
        "Farneback Dense Optical Flow"
    )

# =====
# RUN PART 2
# =====

video_path = "/content/video - Trim.mp4"

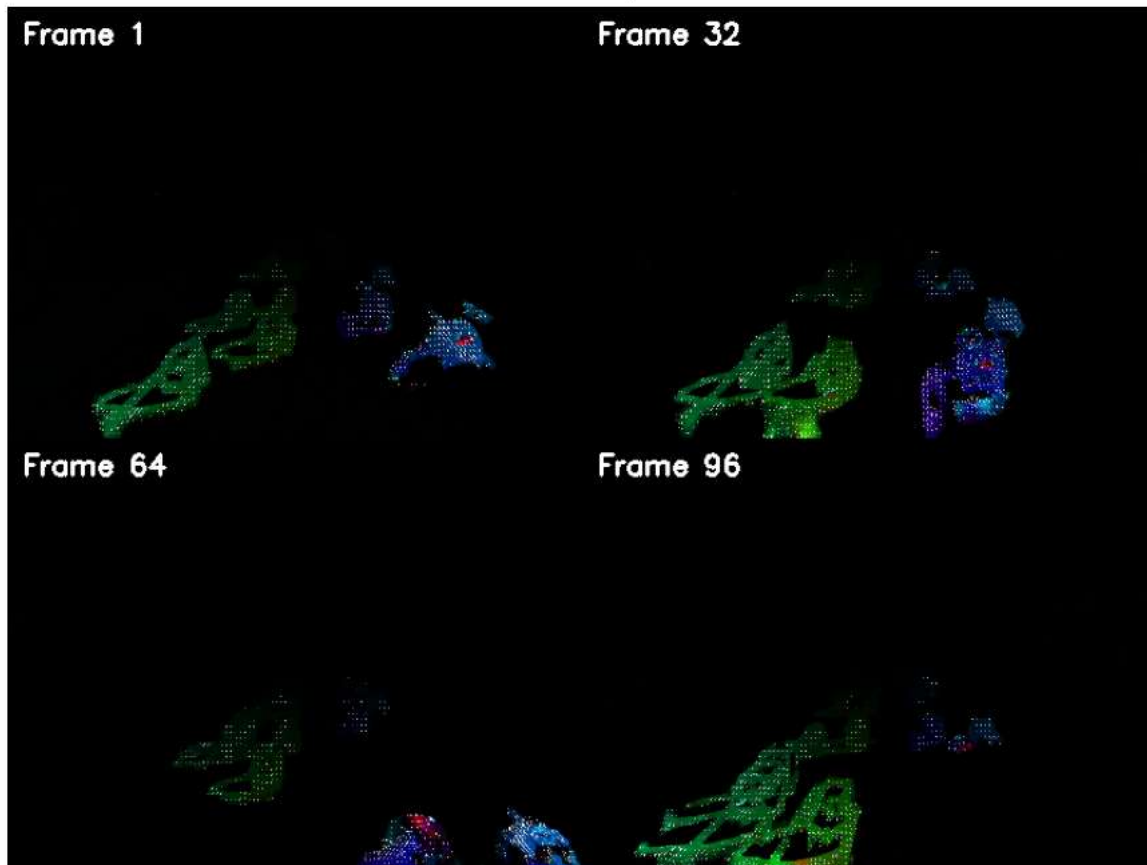
farneback_tracking(video_path)

```

Farneback Optical Flow Processing...

Farneback Completed
Frames processed: 96
Average motion: 1.01

Farneback Dense Optical Flow



✓ Answers to Questions

1. How does object tracking differ from object detection?

Object Detection identifies and locates objects in an image or video frame. It determines what the object is and where it is.

Object Tracking follows an already identified object across multiple consecutive frames and determines its movement and trajectory.

Example:

- **Detection:** Identify a car in a frame.
- **Tracking:** Follow that car as it moves through subsequent frames.

2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow estimates the apparent movement of pixels or feature points between two consecutive video frames.

The basic process is:

- Capture the first frame.
- Detect important feature points.
- Capture the next frame.
- Calculate the movement of those points.
- Draw motion vectors and trajectories.
- Repeat the process for every frame.

Thus, the movement of an object can be continuously tracked in real time. The experiment specifically uses Lucas-Kanade and also asks for comparison with Farneback Dense Optical Flow.

3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi Corner Detection is used to find good feature points in the image.

These points are generally corners or regions with strong intensity changes. Lucas-Kanade then tracks these feature points from one frame to the next.

Therefore:

- **Shi-Tomasi** → Finds good points
- **Lucas-Kanade** → Tracks those points

The supplied experiment specifically instructs selecting feature points using Shi-Tomasi before applying Lucas-Kanade Optical Flow.

4. Why is Optical Flow suitable for motion analysis in videos?

Optical Flow is suitable because it provides information about how pixels or features move between consecutive frames.

It can be used to determine:

- Direction of movement
- Magnitude of movement
- Object displacement
- Object trajectory
- Average speed
- Movement patterns

These parameters are also listed as analysis objectives in the experiment.

5. What challenges arise while tracking fast-moving or partially occluded objects?

Some common challenges are:

- **Fast movement** – An object may move significantly between two frames, making correspondence difficult.
- **Occlusion** – Part of an object may be hidden by another object.
- **Illumination changes** – Changes in lighting can affect feature detection.
- **Motion blur** – Fast objects may become blurred.
- **Camera movement** – Movement of the camera can produce additional optical flow.
- **Loss of feature points** – Previously tracked points may disappear.

The experiment specifically asks evaluation under object speed, illumination changes, occlusion, and camera movement.

6. Compare Optical Flow-based tracking with deep learning-based object tracking.

Optical Flow Tracking	Deep Learning Tracking
Uses pixel/feature movement	Uses trained neural networks
Usually computationally lighter	Usually requires more computational resources
Does not necessarily require a trained object detector	Generally requires training/pre-trained models
Good for motion estimation	Can identify and track specific object classes
Sensitive to illumination and occlusion	Can be more robust when properly trained
Lucas-Kanade and Farneback are examples	Modern detector-tracker architectures are examples

Optical Flow focuses mainly on motion, while deep-learning approaches can combine object recognition and tracking.

7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

Optical Flow can analyze the movement of vehicles, pedestrians, and other objects.

In traffic monitoring, it can help with:

- Vehicle movement analysis
- Traffic-flow estimation
- Speed estimation
- Direction detection
- Congestion analysis

In autonomous driving, optical flow can provide information about:

- Relative motion of objects
- Camera/vehicle movement
- Moving vehicles and pedestrians
- Scene motion
- Possible obstacles

Traffic monitoring and autonomous navigation are among the practical applications mentioned in the experiment.

8. Explain the effect of camera motion on Optical Flow estimation.

Optical Flow normally represents the apparent motion between frames. If the camera itself moves, many or all image points may appear to move even when the objects in the scene are stationary.

For example, if a camera moves to the right, stationary background objects may appear to move to the left.

Therefore, camera motion can create additional flow that makes it difficult to distinguish:

Object motion + Camera motion

from